AI ACADEMY · LEVEL 3 · GRADE 8 · WEEK 22 · STUDENT LESSON

Requests Responses and JSON

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Verified Study Buddy

THE CONNECTED WEEK

Parse the text. Check the schema. Preserve the evidence limit.

Distinguish malformed JSON, invalid task data and unsupported real-world claims.

Keep one habit: Representation table → lunch response → eight cards → guarded Python validator → independent box interface.

MAIN LESSON · SECTION 1

Can we parse this text, and should we accept its values?

> JSON syntax, task schema and factual support are different checks.

Your learning goals

I can parse **JSON** into Python records.

I can validate data after **parsing**.

I can document **input** and **output contract**s.

Recall the tool interface

- Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

- Why is reading text different from validating it?

Your end goal

By the end, I can explain **request**s **response**s and **json** using a traced example and a stated limitation.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

Without opening the old lesson, explain one key idea from Tools and APIs. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from User Need and AI Contribution. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Verify Numbers and Dates. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

Week 22: **Request**s, **Response**s, and **JSON**

Clear **interface**s specify fields, types, errors, and ownership.



CREATOR MISSION

Validate sample **request/response** objects.

Creator name and date:

> **Write or draw your reasoning and evidence.**

Readiness for the connected explanation

This week’s end goal

MAIN LESSON · SECTION 2

Separate text syntax from the service contract

> Reading a value is not accepting it for the task.

Understand the two decisions

JSON is text for exchanging values. Object keys and text values use double quotes; numbers are unquoted, and true, false and null are lowercase. Parsing converts the text into program values. Validation then checks the task’s expected fields, types and ranges. A parser can successfully read an array even when the task requires an object. Neither parsing nor schema validation proves that an observed count is true.

Read the activity-service task

A fictional activity service supplies **JSON** text. The **input contract** is a **JSON object** with name text and minutes an integer from 0 through 60. The **output contract** will be a record with accepted and minutes fields.

Parsing valid **JSON** is only the first step.

▼ **Earlier wording, alternatives and picture transcripts**

PICTURE STORY

Notice → Predict → Explain

Begin with visible **evidence**. Do not explain more than the picture and information support.



MYSTERY

The learners must use **request**s, **response**s, and **json** to solve a problem. What information do they have? What decision or calculation do they make? What would count as **evidence** that it worked?

Three **observation**s (what you can point to):

Write or draw your reasoning and evidence.

My **prediction** and the clue supporting it:

Write or draw your reasoning and evidence.

A closer explanation

MAIN LESSON · SECTION 3

Read the complete activity interface before running code

> The complete guarded implementation is supplied in the investigation.

Closed fields, types, rejection output and policies

Current activity contract: exactly name and minutes. name is text with length greater than zero; this toy rule does not trim whitespace. minutes is an integer from 0 through 60; booleans and decimal-number values are rejected. No extra or duplicate fields are allowed. The output always has accepted and minutes; any rejection returns false and null.

Duplicate rejection is an explicit classroom policy; JSON libraries may otherwise retain a repeated name's last value. Non-JSON NaN/Infinity constants are rejected. These checks are for short supplied classroom cards, not a production network parser.

Translate representations carefully

JSON text	Python value after parsing
null	None
true / false	True / False
"12"	str containing digits
12	int
12.0	float, rejected by this exact-integer classroom schema
{} / []	dict / list; only dict is eligible for this task

Understand parsing before field access

JSON is a text format for exchanging values. **Parsing** asks whether the text follows that format. Schema checking asks whether the parsed value has the structure and values your task requires.

Start by checking that the whole value is an object, represented here by a Python dictionary. Only then look up name and minutes. A valid **JSON** list is still the wrong top-level structure for this **contract**.

Syntax success and acceptance are separate decisions.

▼ Original one-record illustration · limitations retained

```
import json
text = '{"name": "Sketch", "minutes": 12}'
record = json.loads(text)
valid = (type(record) is dict
         and type(record.get("name")) is str
         and len(record["name"]) > 0
         and type(record.get("minutes")) is int
         and 0 <= record["minutes"] <= 60)
result = {"accepted": valid, "minutes": record.get("minutes")}
print(json.dumps(result))
```

Follow the evidence

STEP OR CASE	EVIDENCE	MEANING
Input text	JSON object	Parse with json.loads
Python value	Dictionary fields	Check schema
Output text	accepted and minutes	Serialize with json.dumps

Read each row from left to right and connect the evidence to the stated rule.

The **output** is {"accepted": true, "minutes": 12}. **JSON** uses lowercase true, false and null; Python uses True, False and None. **json**.loads converts between representations.

The worked **output** line assumes the parsed top-level value is a dictionary; a robust **interface** must reject other top-level types before accessing fields.

Explain the deciding check

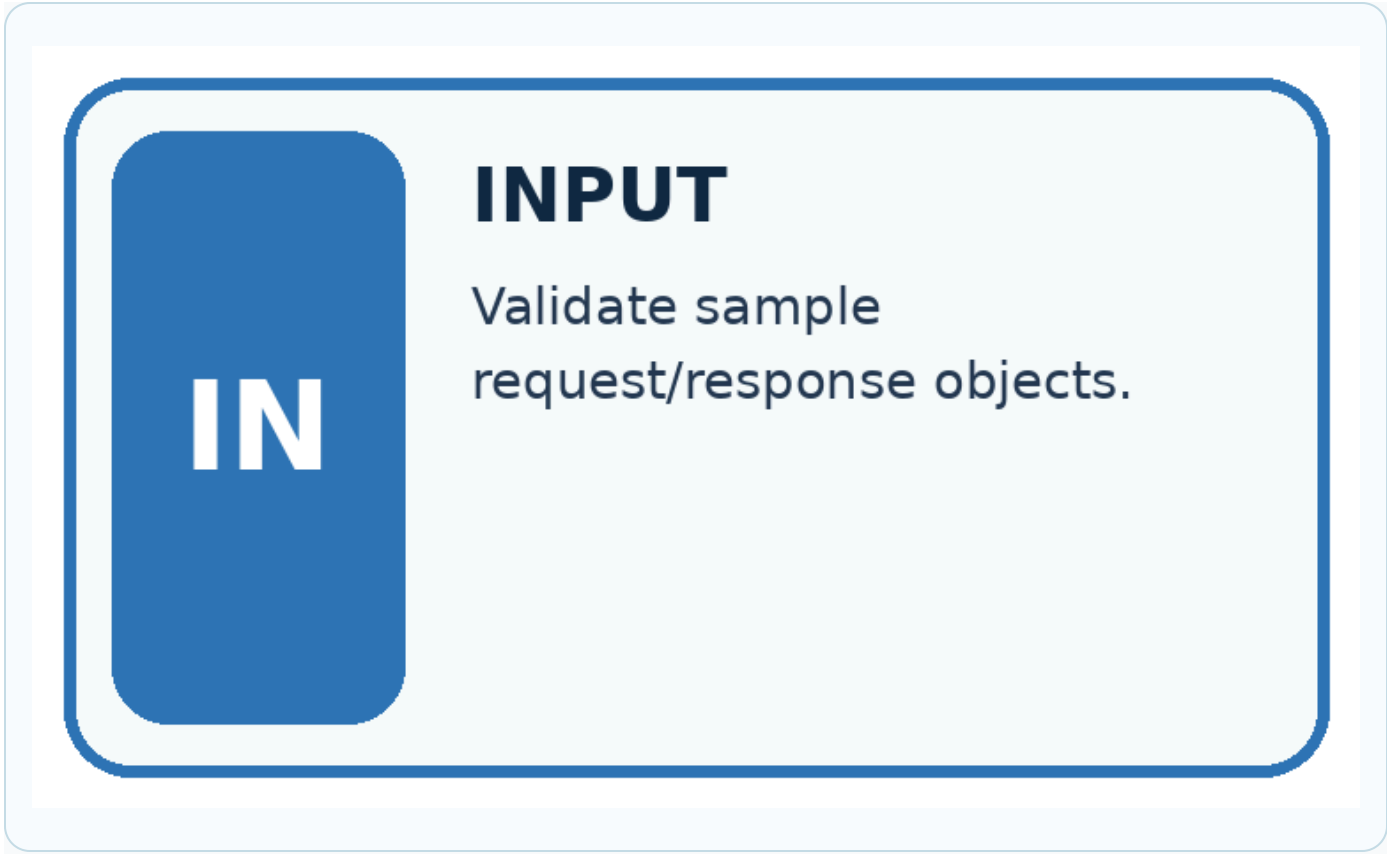
- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ **Earlier wording, alternatives and picture transcripts**

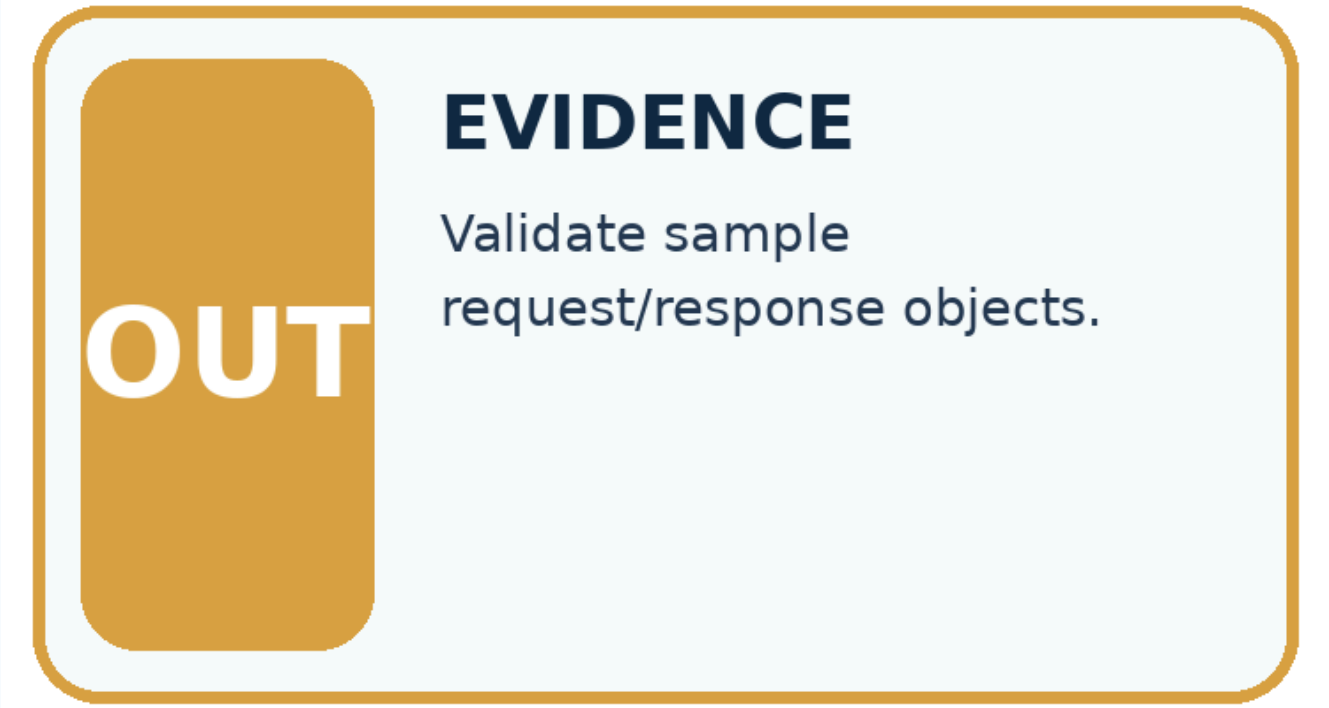
VISUAL **MECHANISM**

Input → **Process** → **Output**

Trace the information. At each stage, ask what changed and how you could test it.


IN
INPUT
Validate sample request/response objects.
↓
SYSTEM
Clear interfaces specify fields, types, errors, and ownership.

↓



Which stage could fail even when the other two are correct? Explain:

> **Write or draw your reasoning and evidence.**

A closer explanation

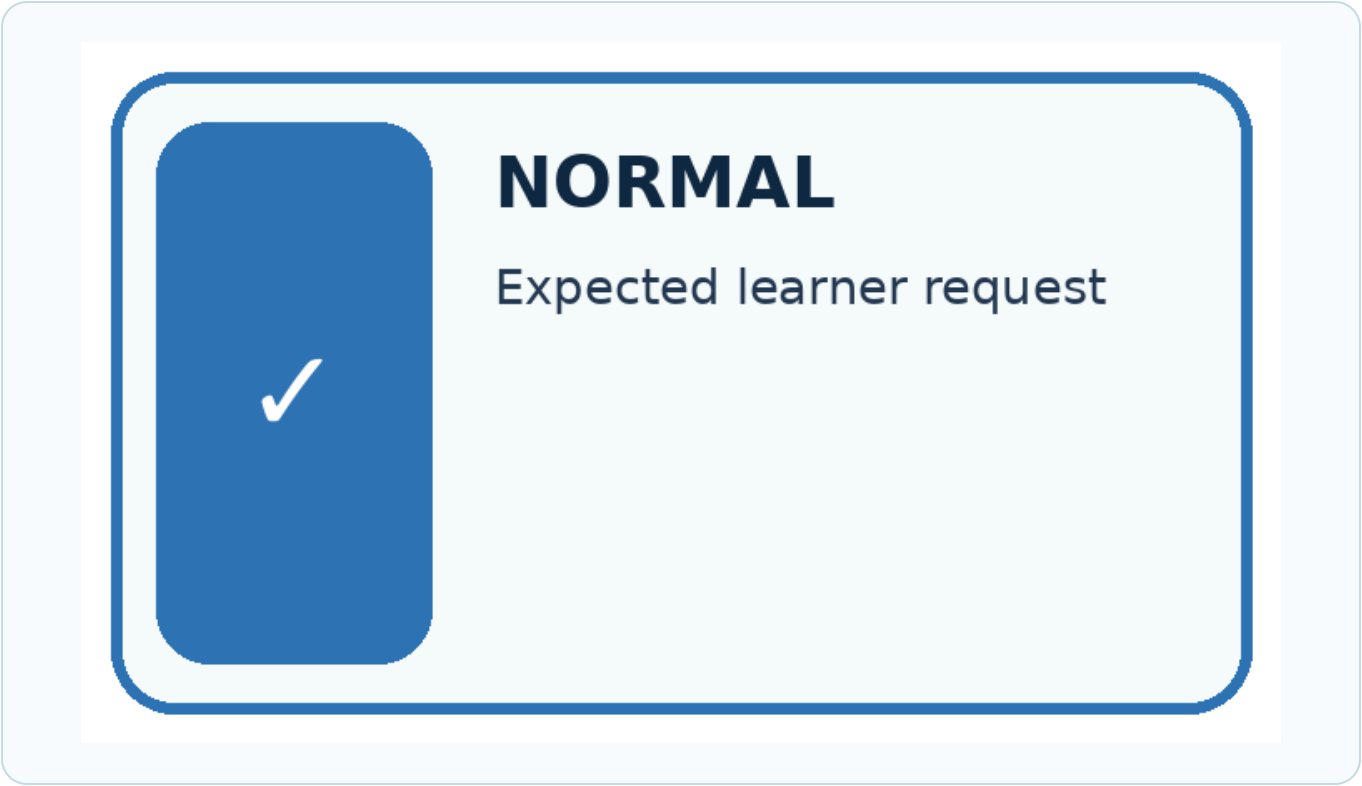
Check the envelope before its fields

Connect the picture and the explanation

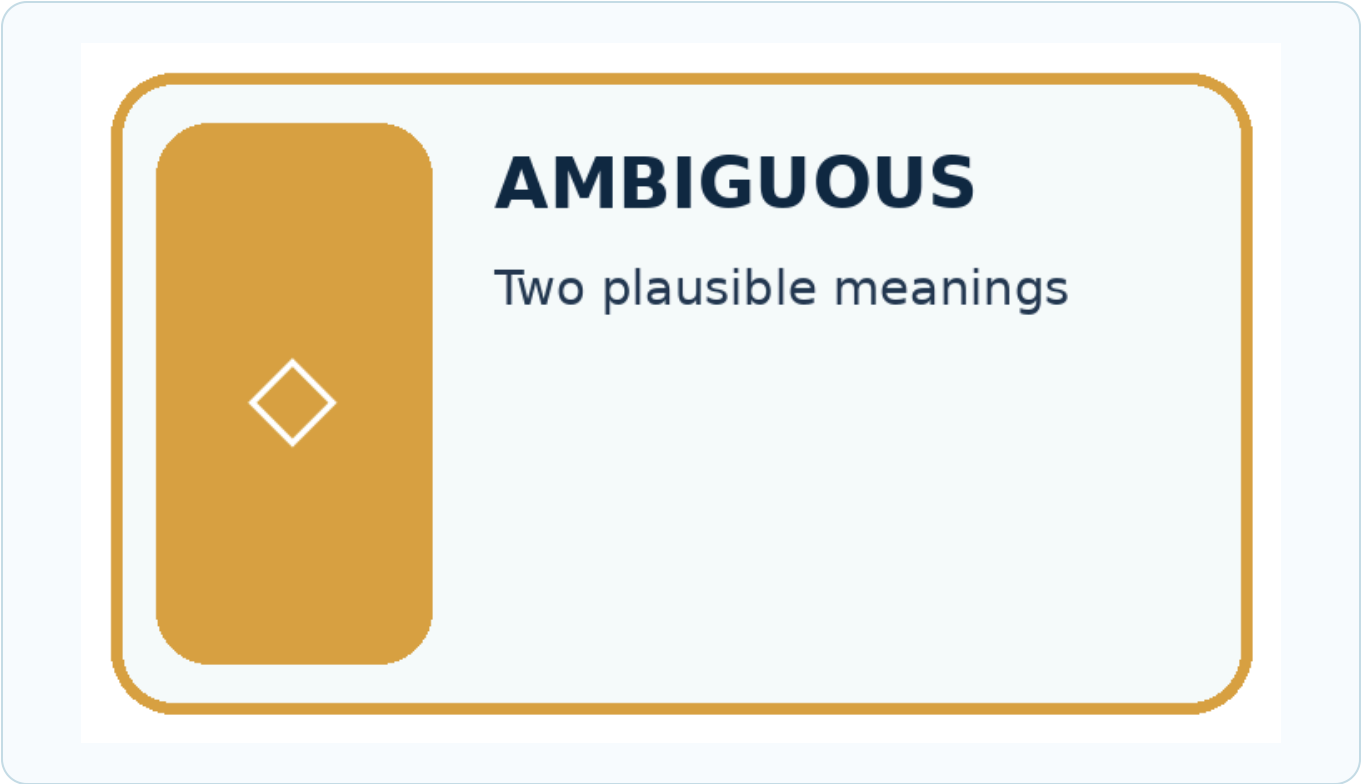
Compare malformed text with valid text of the wrong type

> A valid JSON string value is still not an integer.

Four pictures with a purpose



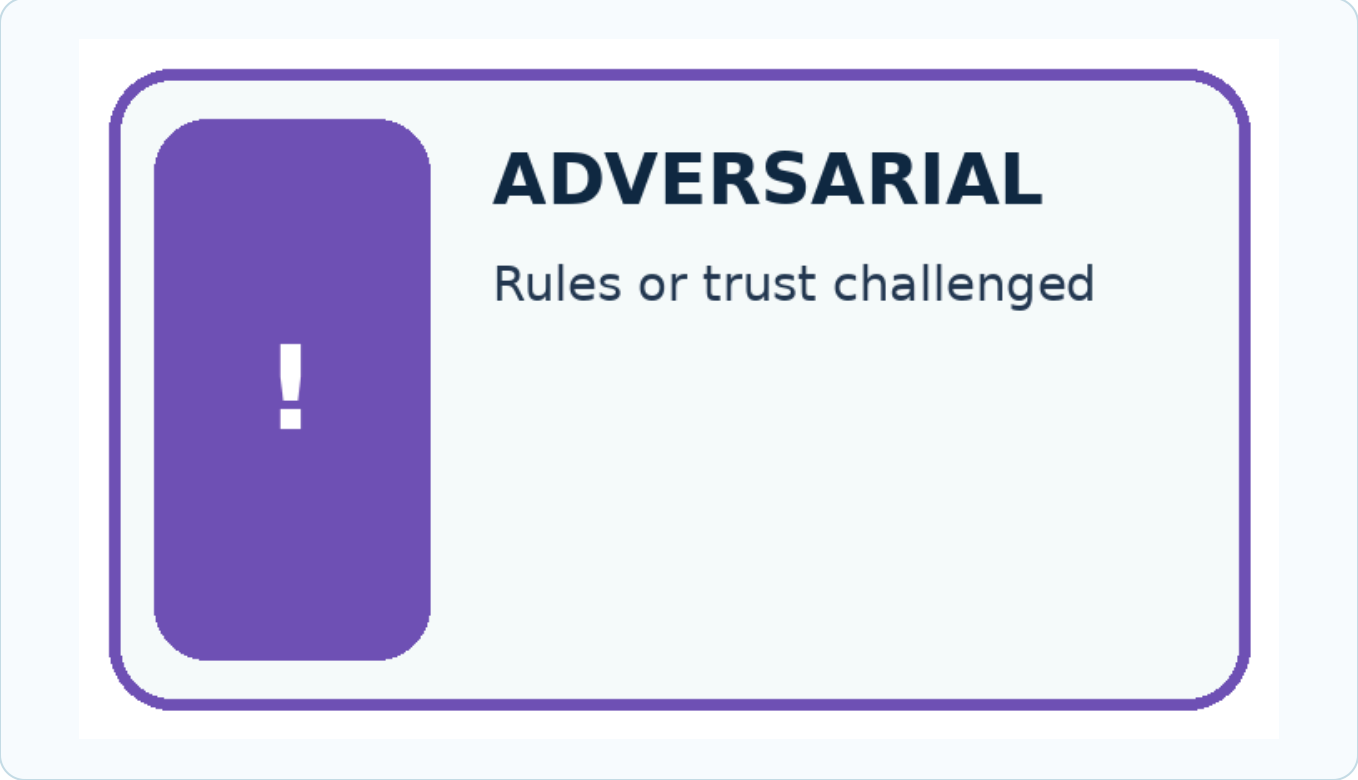
Normal case: predict the result and deciding evidence.



Boundary case: identify the exact limit.



Missing information: name what must be clarified.



Transfer case: explain which procedure still applies.

Choose a revealing case

- Which case is most likely to reveal a hidden assumption? Why?

> **Write or draw your reasoning and evidence.**

Predict the supplied changed case

- Change: Replace 12 with the text "12".

▼ Check after predicting

Prediction: Reject the **response** under this strict **contract**.

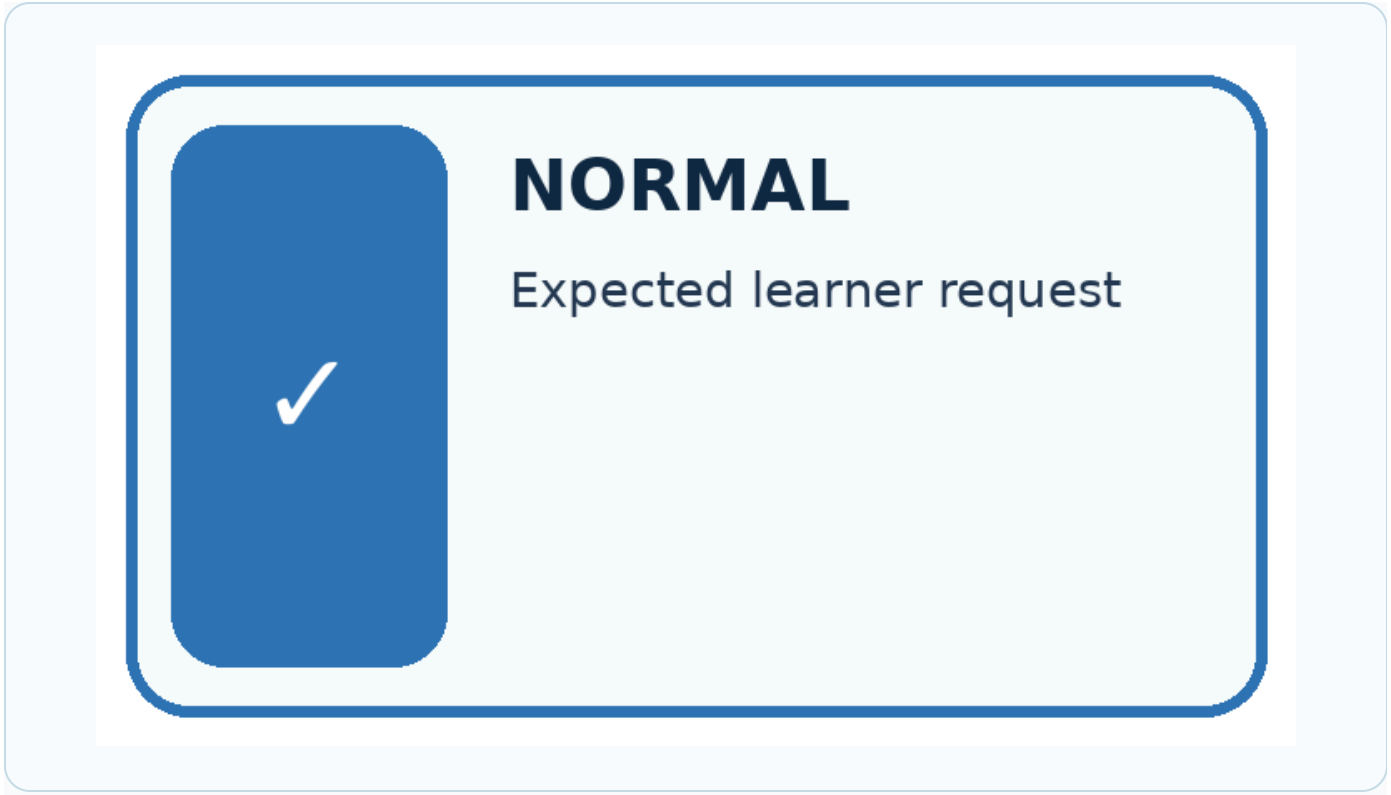
Reason: Digits inside quotes represent text, whereas portions requires an integer.

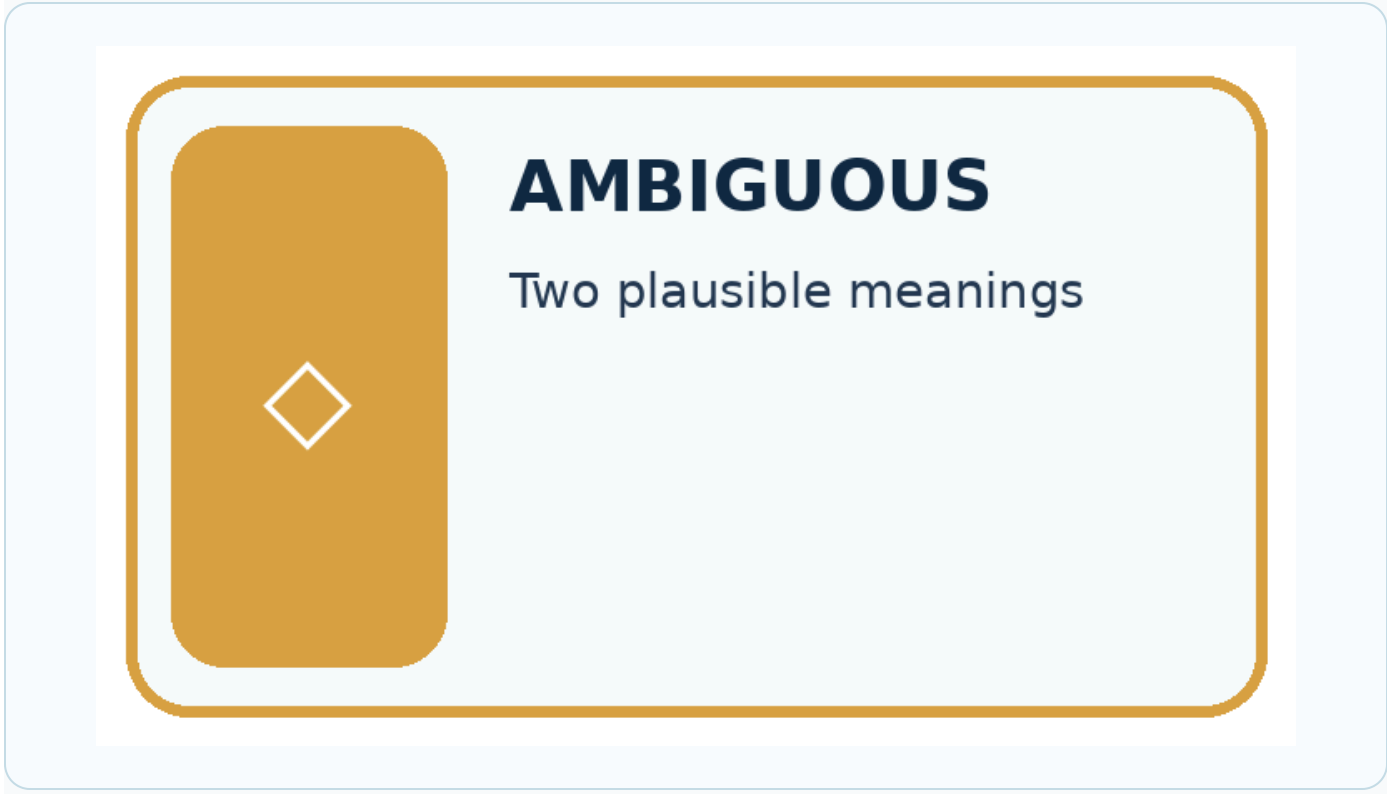
▼ Earlier wording, alternatives and picture transcripts

CASE GALLERY

Four Tests, Four Kinds of **Evidence**

A system is not understood until it survives more than one comfortable example.





NORMAL CASE: predict and name **evidence**. BOUNDARY CASE: predict and name **evidence**.





MISSING INFORMATION: predict and name **evidence**. TRANSFER CASE: predict and name **evidence**.

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Trace the lunch response without silently converting it

> The contract determines whether quoted digits are acceptable.

Read the complete lunch scope

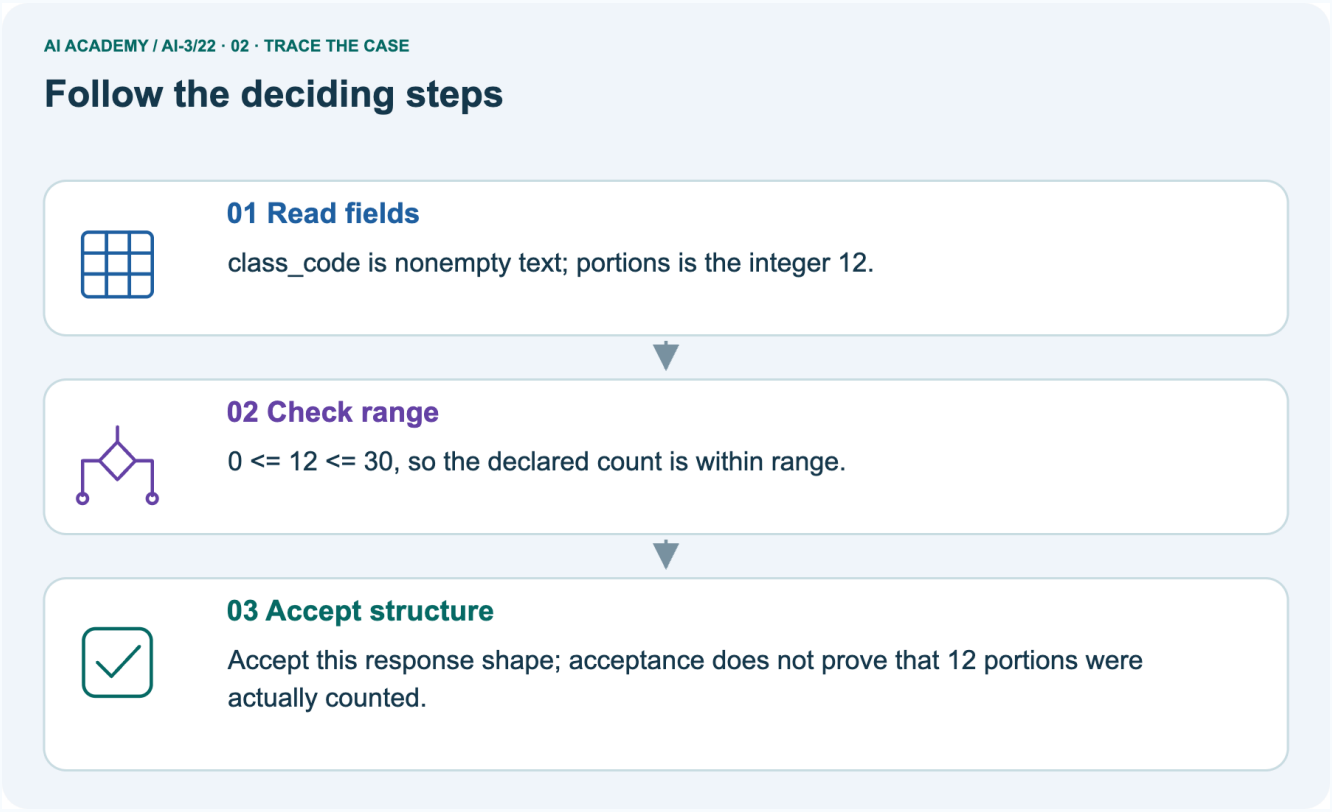
A lunch-count **response** must satisfy a declared **JSON contract**.

The toy **response** has exactly class_code text and portions integer 0..30. Example: {"class_code":"R8","portions": 12}.

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

Separate lunch-response contract: exactly class_code as nonempty text and portions as an integer from 0 through 30. The example R8/12 satisfies that schema; the changed value "12" is text and fails. Accepting a structure does not verify an actual lunch count or authorise an order. Keep the activity service’s name/minutes fields separate from this schema.

Trace fields, range and structural acceptance



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Read fields	class_code is nonempty text; portions is the integer 12.
2. Check range	0 <= 12 <= 30, so the declared count is within range.
3. Accept structure	Accept this response shape; acceptance does not prove that 12 portions were actually counted.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

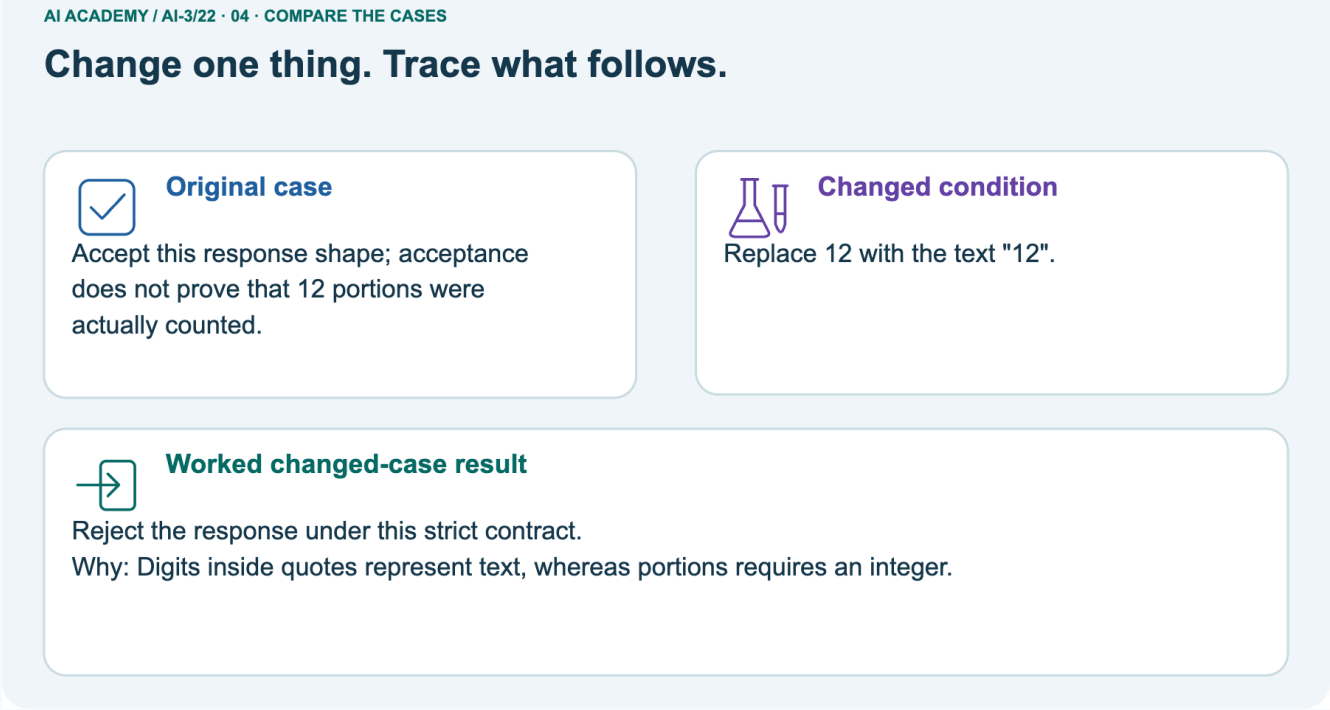
Read the picture in order. Accept this **response** shape; acceptance does not prove that 12 portions were actually counted. The arrows show which recorded **input** or intermediate result each decision uses.

Explain the deciding step aloud before checking the changed case; pointing to the final answer alone does not show how it follows.

Predict replacing the integer with text

- Change one thing: Replace 12 with the text "12". Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

Name the factual-evidence limit

- **JSON** well-formedness, schema validity and real-world truth are different checks.
- Read the labelled picture: Would converting the text silently prove the source count is correct? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

▼ Additional practice · Earlier request/response outline

1. **Request** requires question:text and source_ids:list.
2. **Response** requires answer:text, citations:list, **status**:text.
3. Missing citations or wrong source_ids type fails validation.

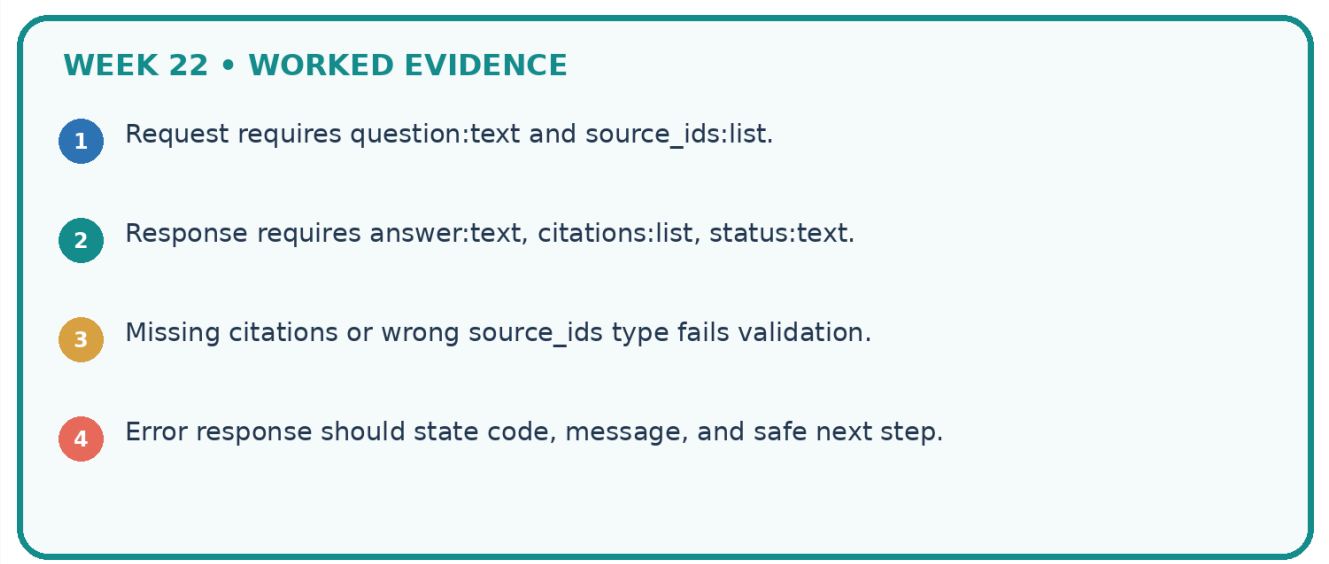
4. Error **response** should state code, message, and safe next step.

▼ Earlier wording, alternatives and picture transcripts

WORKED **EVIDENCE**

Reason Step by Step

Predict first. Then check each step. A correct answer without visible reasoning is incomplete **evidence**.



Recalculate, retrace, or restate the reasoning in your own representation:

Write or draw your reasoning and evidence.

CHECK Did the **evidence** answer the exact question? Did the total, path, or source record stay consistent?

A closer explanation

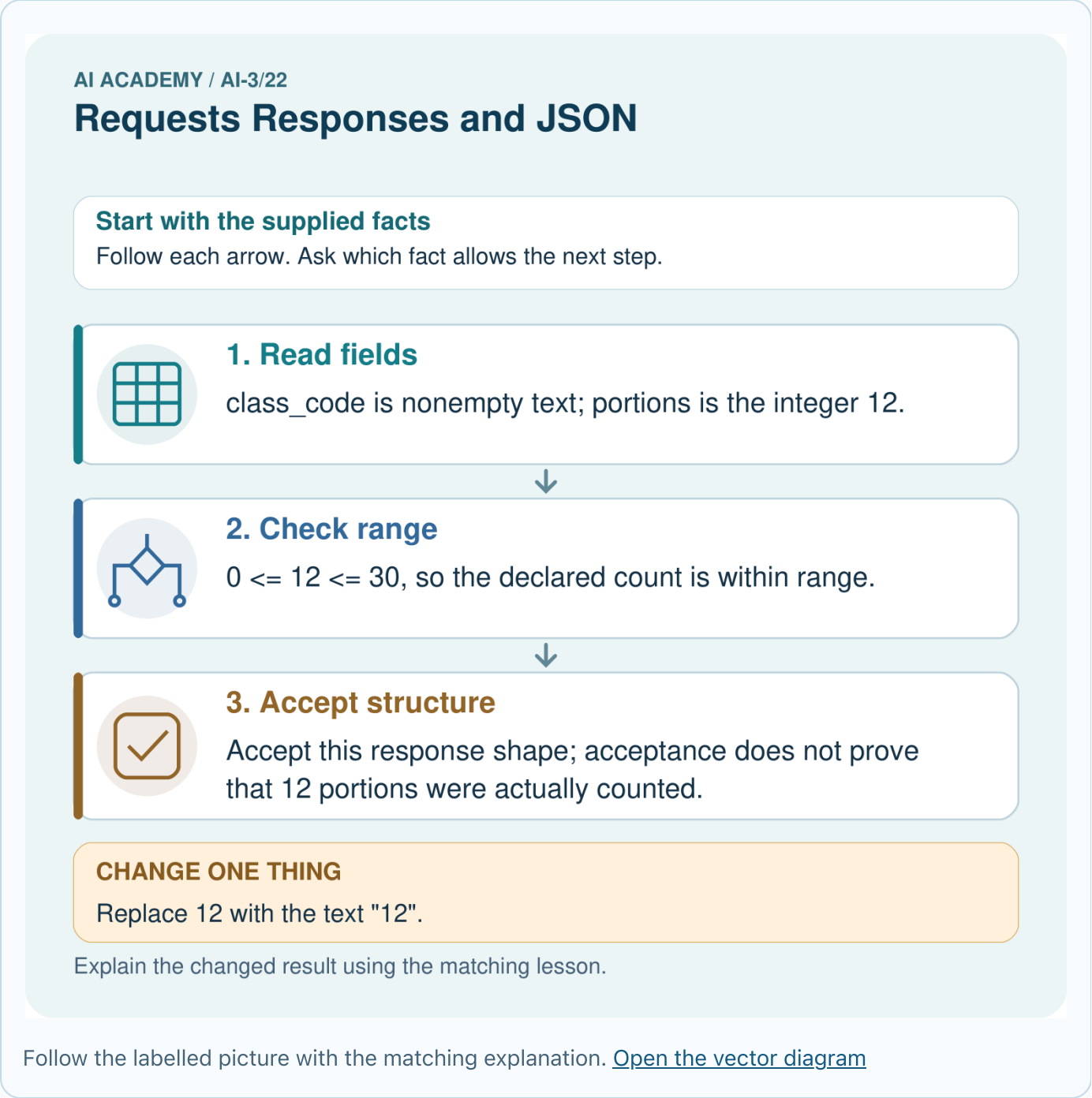
> **JSON text contains {"name": "Sketch", "minutes": "12"}. Does parsing fail, or does validation fail?**

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Requests Responses and JSON

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Find a successful parse mistaken for a valid record

> Validate the whole value before accessing fields.

Repair the field-access shortcut

Claim to repair: The parser returned a value, so call .get on it and display the minutes. A parsed list, null or number has the wrong top-level structure. Check for a dictionary first, then exact fields, types and ranges. A boolean is not an integer count under this strict schema, even though Python bool is a subclass of int.

Reject "12" rather than silently converting it; conversion would be a different declared contract.

Design a revealing follow-up

> **A JSON record parses successfully. Does that prove its observation is correct?**

▼ **Earlier wording, alternatives and picture transcripts**

ERROR DETECTIVE

Compare → Diagnose → Repair

Do not patch the answer. Find the earliest unsupported assumption, wrong step, or weak **evidence**.

!

COMPARE • DIAGNOSE

Weak idea: Valid JSON syntax proves the values are meaningful and safe. Strong move: inspect the claim, source, stage, schema, or test evidence; repair one cause; rerun.

Evidence record	Expected	Actual	Likely cause / next test
normal case	works		
boundary case	stated rule		
missing input	safe response		
fresh transfer	generalizes		

Repair one cause. Why is this repair better than changing several things?

> **Write or draw your reasoning and evidence.**

A closer explanation

An **interface** should state what happens for malformed text, wrong structure and invalid values. Do not expose raw private **input** in error messages. Use only the fictional records supplied for practice.

Practise syntax and schema failures with less help

> The W2 strings isolate distinct failure stages; the missing name is also invalid under the complete activity schema.

Recognise JSON quoting

Single quotes around property names are not valid **JSON** syntax. Use double quotes in the **JSON** text. If the whole parsed value is a list, do not call .get on it: check the top-level type first and report a schema failure.

Complete the syntax/type W2

Use workbook W2 for the connected example "**JSON** and **interface contract**s". First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Audit a successful-run claim

- Use the worked case for **JSON** and **interface contract**s. Proposed explanation: "A successful run proves the conclusion". Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the pictured strict type rule

- Would converting the text silently prove the source count is correct? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

My explanation using a pictured step and the given facts

Keep independent work separate

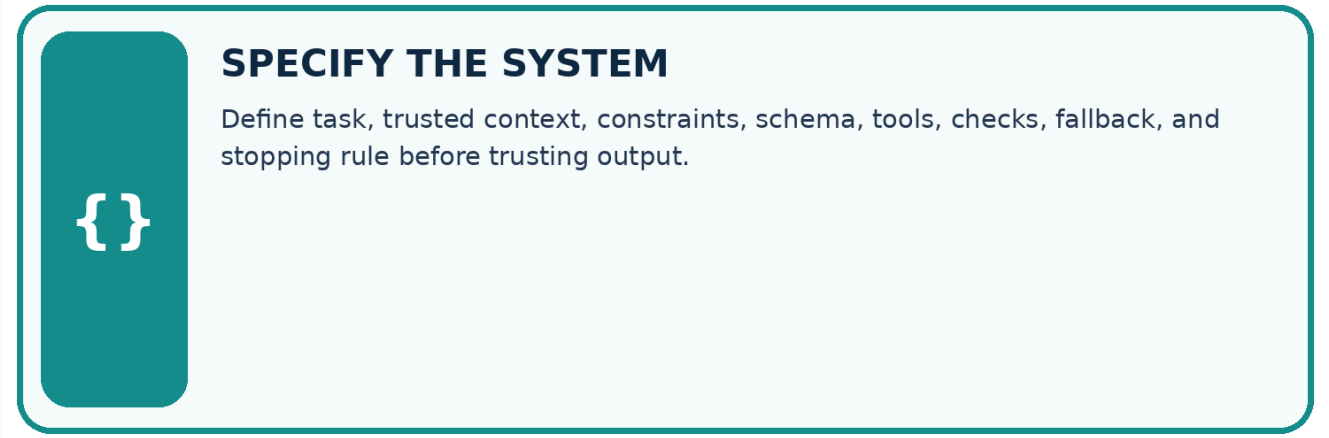
This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ Earlier wording, alternatives and picture transcripts

LOGIC STUDIO

Specify Before You Run

Prompts, schemas, retrieval, tool calls, and code are inspectable parts of one AI product.



SET task, audience, constraints, **output**_schema LOAD approved_context or retrieval_result CALL model or allowed_tool VALIDATE fields, claims, citations, and safety IF **evidence** missing: ask, retrieve, or abstain RETURN useful **output** + source trail + limitations

Trace one **input**. Which line changes the state or chooses a path?

Write or draw your reasoning and evidence.

A closer explanation

Practise with less help: Separate failures

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Inspect eight complete records and run the guarded validator

> A safe rejection has an explicit output and no fabricated measurement.

Predict the eight supplied outcomes

Card	Supplied JSON text	First relevant outcome
1	{"name":"Sketch","minutes":12}	accepted, minutes 12
2	{name:"Sketch","minutes":12}	syntax: unquoted name
3	[]	schema: top-level object
4	{"name":"Sketch"}	schema: missing minutes / exact fields
5	{"name":"Sketch","minutes":"12"}	schema: integer minutes

Card	Supplied JSON text	First relevant outcome
6	{"name":"Sketch","minutes":true}	schema: integer minutes
7	{"name":"Sketch","minutes":61}	schema: minutes range
8	{"name":"Sketch","minutes":0}	accepted, legitimate zero

Choose paper trace or complete Python run

1. Predict the first failing stage for each complete card before reading the outcome column. Trace on paper or run the complete program; label which you did. On rejection, output accepted=false and minutes=null, not an invented zero. Repair a syntax error only when the intended text is supplied; missing factual values require the source owner. Add a duplicate-field, non-object or boundary test and predict its result.

   A strict schema can reject a record without disproving its underlying observation. No service call or private input is required. Keep the independent label/count contract unused until its first attempt.

Complete validator and supplied cards

```
import json

class DuplicateField(ValueError):
    pass

def unique_object(pairs):
    result = {}
    for key, value in pairs:
        if key in result:
            raise DuplicateField('duplicate field')
        result[key] = value
    return result

def reject_constant(token):
    raise ValueError('non-JSON numeric constant')
```

```
def validate_activity(text):
    # This lesson supplies short text cards, not unbounded network input.
    rejected = {'accepted': False, 'minutes': None}
    try:
        record = json.loads(text, object_pairs_hook=unique_object,
                            parse_constant=reject_constant)
    except DuplicateField:
        return 'duplicate-field policy', rejected
    except (json.JSONDecodeError, ValueError):
        return 'syntax', rejected
    if type(record) is not dict:
        return 'schema: top-level object', rejected
    if set(record) != {'name', 'minutes'}:
        return 'schema: exact fields', rejected
    if type(record['name']) is not str or len(record['name']) == 0:
        return 'schema: name', rejected
    if type(record['minutes']) is not int:
        return 'schema: integer minutes', rejected
    if not 0 <= record['minutes'] <= 60:
        return 'schema: minutes range', rejected
    return 'accepted', {'accepted': True, 'minutes': record['minutes']}

cards = [
    '{"name":"Sketch","minutes":12}',
    '{name:"Sketch","minutes":12}',
    '[]',
    '{"name":"Sketch"}',
    '{"name":"Sketch","minutes":"12"}',
    '{"name":"Sketch","minutes":true}',
    '{"name":"Sketch","minutes":61}',
    '{"name":"Sketch","minutes":0}',
]
for number, text in enumerate(cards, 1):
    status, result = validate_activity(text)
    print(number, status, json.dumps(result, allow_nan=False))
```

Keep the evidence trail

Test / input · Prediction · Actual · Evidence / calculation · Change or keep?

> **Test / input:** normal
>
> **Prediction:** _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: boundary

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: missing

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

Test / input: fresh transfer

Prediction: _____

Actual: _____

Evidence / calculation: _____

Change or keep?: _____

What did the hardest test teach you that the normal test did not?

Write or draw your reasoning and evidence.

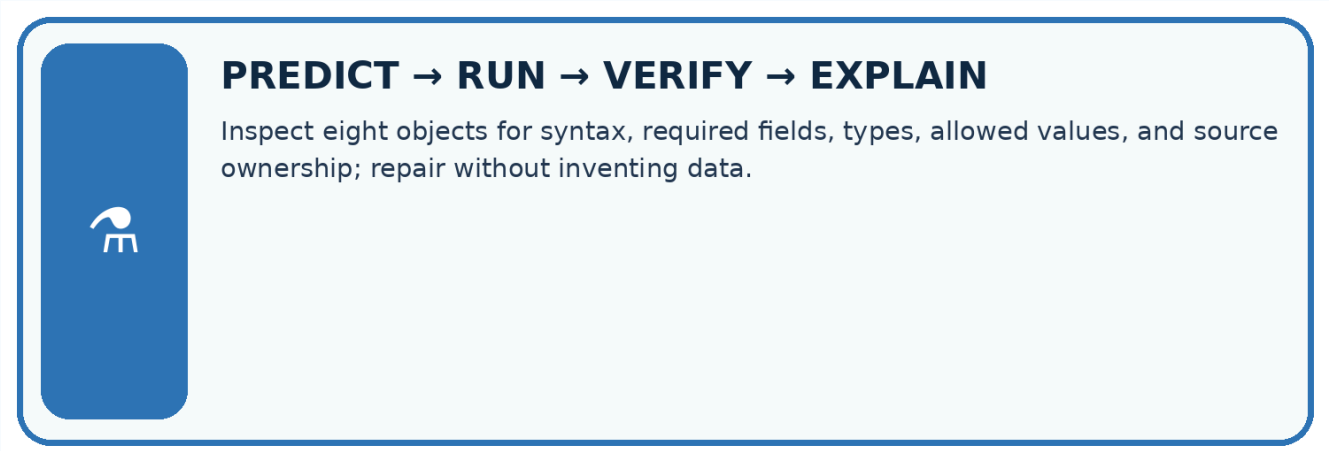
Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

EVIDENCE LAB

Predict → Test → Record → Explain

Keep expected and actual results separate. Surprises are useful data when recorded honestly.



Use the eight supplied activity JSON cards and complete guarded validator. Separate syntax, top-level structure, exact fields, types and range errors; preserve a real zero and return null on rejection. Use paper traces or recorded Python execution, not invented service responses.

Plan → try → check
</details>

MAIN LESSON · SECTION 14

Try alone: specify the fresh label/count contract

> Use the task’s fields and limits rather than copying activity-service names.

Independent specification and scope

Define a **JSON contract** for {"label": "Box", "count": 4}: nonempty text label, integer count 0 through 10. Give a valid case, a wrong-type case and a missing-field case with expected acceptance.

Specify your choices explicitly for missing or extra fields and the output on rejection. The task supplies field meanings and bounds, but no real box inventory. Treat acceptance as a contract decision rather than evidence that a box was physically counted.

Your first attempt and help record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Your teacher has the independent answer and scoring criteria. Complete the matching workbook week to keep your practice **evidence** together.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Specify independently

Week 22 Independent application

Close the worked explanation. Apply the idea to this different question without copying.

An API returns {"ok":false,"error":"not_found"}. Your app tries to display **response**.title. What should it do instead?

My answer, with a trace, calculation or supporting **evidence**

> **Write or draw your reasoning and evidence.**

Create and test

Change one relevant detail. Predict the result before testing. Include a boundary or missing-information case when it helps reveal a limit.

My changed **input** and expected result

> **Write or draw your reasoning and evidence.**

Connect to your project

Use a stable **request/response contract** between **interface**, retriever, model, and verifier.

The **evidence** I will keep

Write or draw your reasoning and evidence.

Transfer practice from the original programme

Request id=7 returns **status**=200 and {"id":8,"pages":90}. Explain why a successful **status** is insufficient and specify the handling.

Explain the mechanism, evidence and one limit.

Use feedback to improve

MAIN LESSON · SECTION 11

Keep the project's interfaces predictable

> Every component needs a defined success and error path.

This week's contribution

For the Verified Study Buddy, document each interface's owner, exact request/response schema, success output, rejection status and missing-data behavior. Validate parsed structure before field access, then check task semantics and source identity. Use fictional inputs in demonstrations and concise error codes rather than echoing private raw input. Record schema acceptance separately from verified factual support.

Retain the checkpoint and human choice

THIS WEEK'S CHECKPOINT Use a stable **request/response contract** between **interface**, retriever, model, and verifier.

SYSTEM MAP Learner need → prompt/schema → approved sources/retrieval → model/tool → structured answer → verification → **human decision**

What will you add, change, or verify in the project this week?

> **Write or draw your reasoning and evidence.**

What should remain under **human control**?

> **Write or draw your reasoning and evidence.**

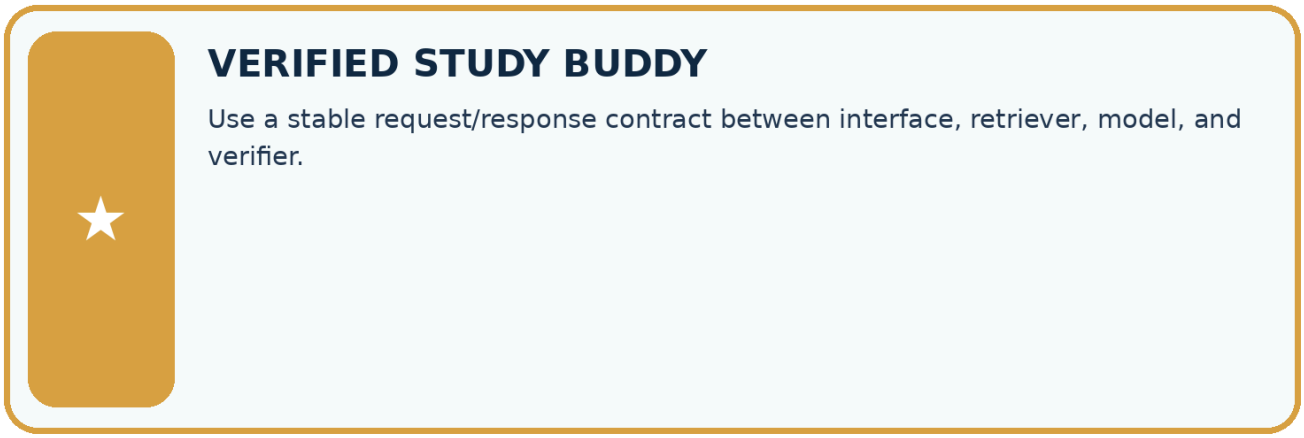
RESPONSIBLE DESIGN Collect only necessary information. Show uncertainty and limits. Never confuse fluent **output** with verified truth.

▼ Earlier wording, alternatives and picture transcripts

CUMULATIVE PROJECT

Verified Study Buddy

Every week adds one inspectable piece. The system must show how it reached a result.



Evidence for your project

For your Verified Study Buddy, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding.

MAIN LESSON · SECTION 16

Finish: distinguish readable, valid and verified

> A successful parse cannot establish the truth of an observation.

Explain without notes

- Close your notes. Explain this goal in your own words: Parse **JSON** into Python records

My deciding step, evidence and one remaining question

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Your quick word guide**

REFERENCE · SECTION 5

Your quick word guide

> Use the words to explain a particular step, record or limit.

Core vocabulary

Word	My own example	Not this / boundary
request	A message asking a system to return information or perform an action.	
response	The information or status returned after a request.	
JSON object	A collection of named fields and values written using JSON syntax, such as {"pages":80}.	
status	A reported condition of a request or process, such as success, pending or error.	

Terms for the connected explanation

Word	Meaning
JSON	A text format for structured values
Parsing	Converting formatted text into program values

Word	Meaning
Interface	The agreed way one part of a system exchanges data with another
Contract	Specified inputs, outputs and error behavior

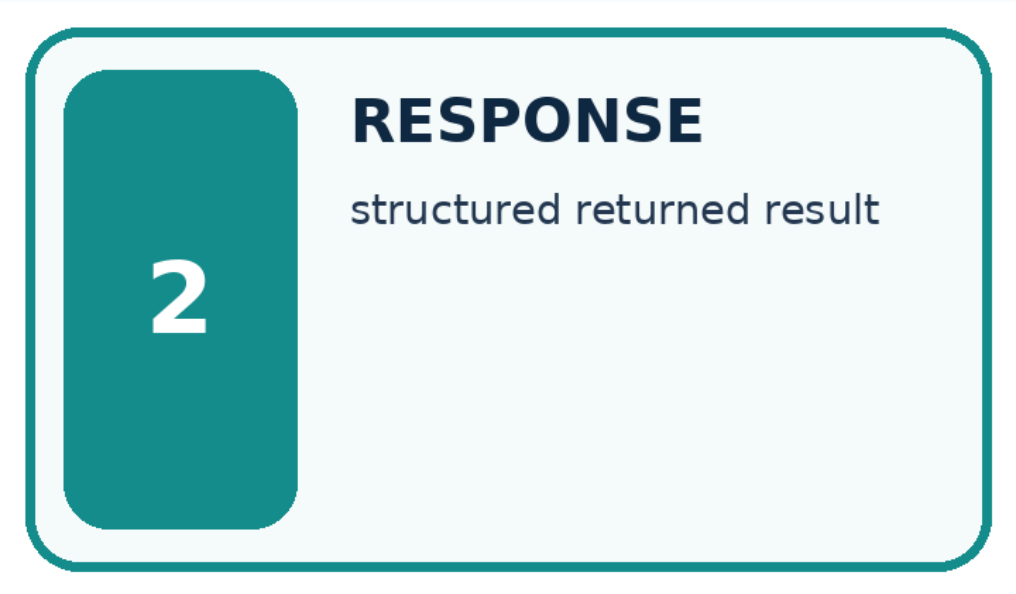
▼ Earlier wording, alternatives and picture transcripts

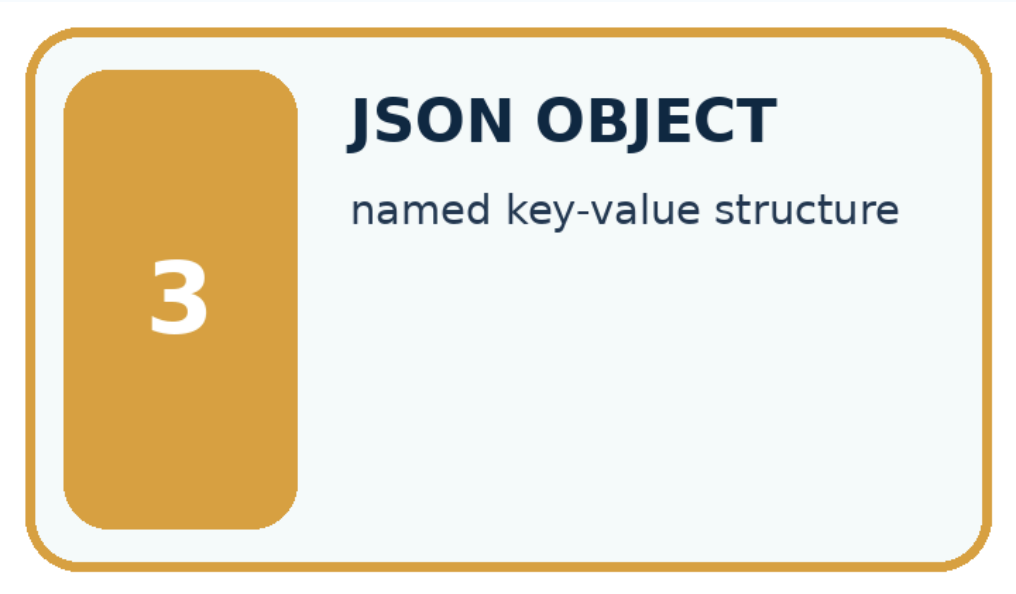
VOCABULARY LAB

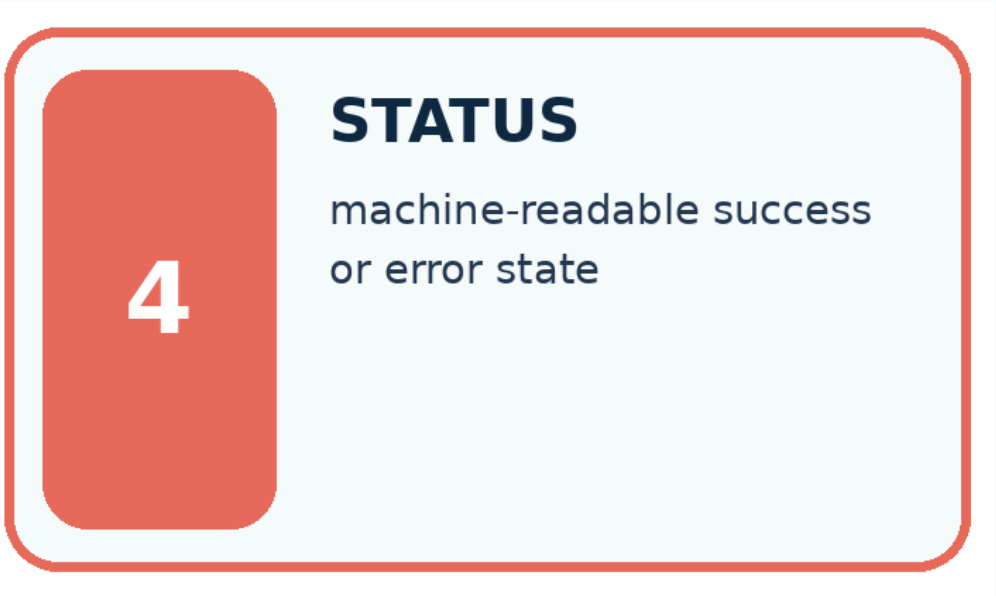
Words You Can Use to Reason

A definition matters when you can recognize the idea, use it, and contrast it with a near-miss.




2
RESPONSE
structured returned result


3
JSON OBJECT
named key-value structure



Words used in the closer explanation

▼ **Choose a stronger thinking challenge**

REFERENCE · SECTION 10

Choose a stronger thinking challenge

> Choose the next useful challenge rather than treating the whole ladder as compulsory.

Pick your next step

- 1 • NAME: Define the concept without repeating the textbook.
- 2 • TRACE: Show the information, state, branch, calculation, or **evidence** path.
- 3 • APPLY: Solve a different example independently.
- 4 • CHALLENGE: Find a boundary case, counterexample, or unfair outcome.

- 5 • EXPLAIN LIMITS: Say when the result should not be trusted or used alone.

- My strongest **evidence** of independent understanding:

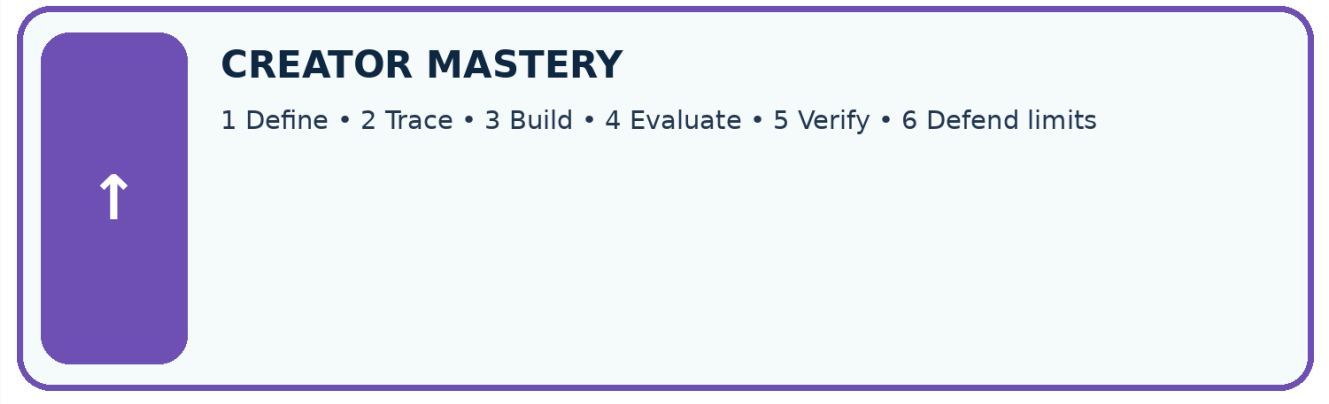
Write or draw your reasoning and evidence.

▼ Earlier wording, alternatives and picture transcripts

MASTERY

From Knowing the Word to Owning the Idea

Move upward only when you can produce **evidence** without copying the worked example.



▼ Reference: check your explanation

Reference: check your explanation

> Use after practice. Specify the new label/count interface independently is the selected independent case.

Check the evidence and its limit

- 1. My fresh example, trace, calculation, or design:
- 2. My **evidence** that it works (include a boundary or missing-data case):
- 3. One limitation, fairness concern, or situation needing human review:

> **Write or draw your reasoning and evidence.**

> **Write or draw your reasoning and evidence.**

Self-check and record

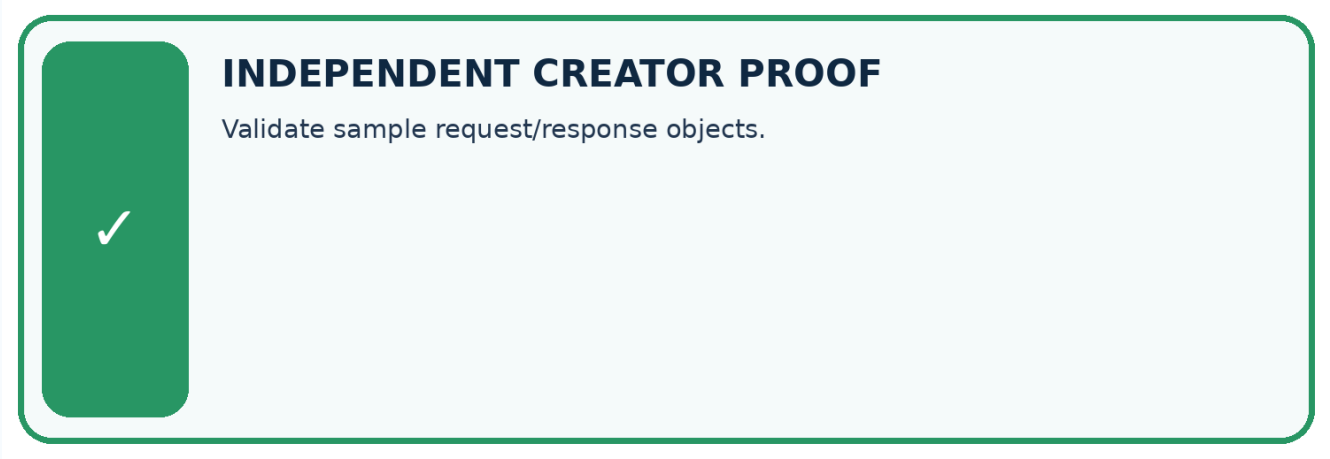
SELF-CHECK ☐ I explained the **mechanism** ☐ I used **evidence** ☐ I tested a fresh case ☐ I named a limitation ☐ I can answer 'why?'

▼ **Earlier wording, alternatives and picture transcripts**

INDEPENDENT MASTERY PROOF

Checklist for explaining this week's idea

Use after practice. Specify the new label/count interface independently is the selected independent case.



EXPLANATION CHECK Require exact types and bounds, explicit missing-field handling and separation from factual verification.

> **Write or draw your reasoning and evidence.**

▼ **Extra practice: Validate the API response before using it**

REFERENCE · SECTION 13

Extra practice: Validate the API response before using it

> Try the question before opening its worked explanation.

Read the supplied case and question

Request is for B7; **response** is {"id":"B7","pages":120}. Another **response** has malformed **JSON**.

What checks precede use?

Your explanation

- Which detail supports the answer, and why does it matter?

Write or draw your reasoning and evidence.

▼ Check after trying

Request success, **JSON** validity, matching id, required fields and numeric type. Handle malformed **response**s explicitly.

Evidence or stage	Value or result
Request success	Handle failure explicitly
JSON syntax	Reject malformed data
Identity	id must match B7
Field type	pages must be numeric 120

▼ Earlier wording, alternatives and picture transcripts

Week 22 Worked case

Use the supplied details to practise the weekly concept before attempting the independent question. This case extends the illustrated lesson.

The situation

The question

Worked explanation

Explain it in your own words

Trace the supplied case visually

Validate the API response before using it

Request is for B7; **response** is {"id":"B7","pages":120}. Another **response** has malformed **JSON**.

What checks precede use?

Request success, **JSON** validity, matching id, required fields and numeric type. Handle malformed **response**s explicitly.

▼ Reference: original lesson checklist

REFERENCE · SECTION 15

Reference: original lesson checklist

> Use an additional checklist when it helps inspect your explanation.

Targets and evidence

By the end, I can	How I will show it
Parse JSON into Python records	Explain the deciding step, show a trace or test, and state one limit.
Validate data after parsing	Explain the deciding step, show a trace or test, and state one limit.
Document input and output contracts	Explain the deciding step, show a trace or test, and state one limit.

Keep your route and first prediction

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: A lunch-count **response** must satisfy a declared **JSON contract**.

▼ Extra practice: pictures, repair and feedback

REFERENCE · SECTION 17

Extra practice: pictures, repair and feedback

> Attempt each question before opening its explanation. Use one browser or paper copy.

First supported question

- A lunch-count **response** must satisfy a declared **JSON contract**.

- Which labelled step matches this detail? Accept this **response** shape; acceptance does not prove that 12 portions were actually counted.
- 1. Accept structure
- 2. Read fields
- 3. Check range
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.
- After your attempt, compare with the supported explanation: Accept structure Accept this **response** shape; acceptance does not prove that 12 portions were actually counted. Read the picture in order.

  Accept this **response** shape; acceptance does not prove that 12 portions were actually counted. The arrows show which recorded **input** or intermediate result each decision uses.

  Explain the deciding step aloud before checking the changed case; pointing to the final answer alone does not show how it follows.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ Check the first explanation

Practice 2: explain the deciding evidence

Changed-case question

- Changed-case practice: Replace 12 with the text "12". Which conclusion is supported by the supplied facts?
- 1. This one example guarantees correct results for every future **input**.
- 2. We can decide the result without examining the changed **input** or the procedure.
- 3. Reject the **response** under this strict **contract**.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.
- After your attempt, compare with the supported explanation: Reject the **response** under this strict **contract**. Digits inside quotes represent text, whereas portions requires an integer.

My selected conclusion, deciding fact and explanation

▼ Check the changed case

Repair a missing building block

Repair and save your work

Review Tools and APIs (ai-3/21). Goal: Explain the **mechanism** and terminology of tools and api **contract**s. Short practice: Does a valid C7 ID permit the extra lending action?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review User Need and AI Contribution (ai-3/19). Goal: Explain the **mechanism** and terminology of capstone requirements and corpus. Short practice: What non-AI baseline would reveal whether this helper adds value?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review Verify Numbers and Dates (ai-3/16). Goal: Verify a numeric claim using its source, denominator and units. Short practice: Which part can be corrected immediately from existing **evidence**, and which needs follow-up?. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Predict, run and debug

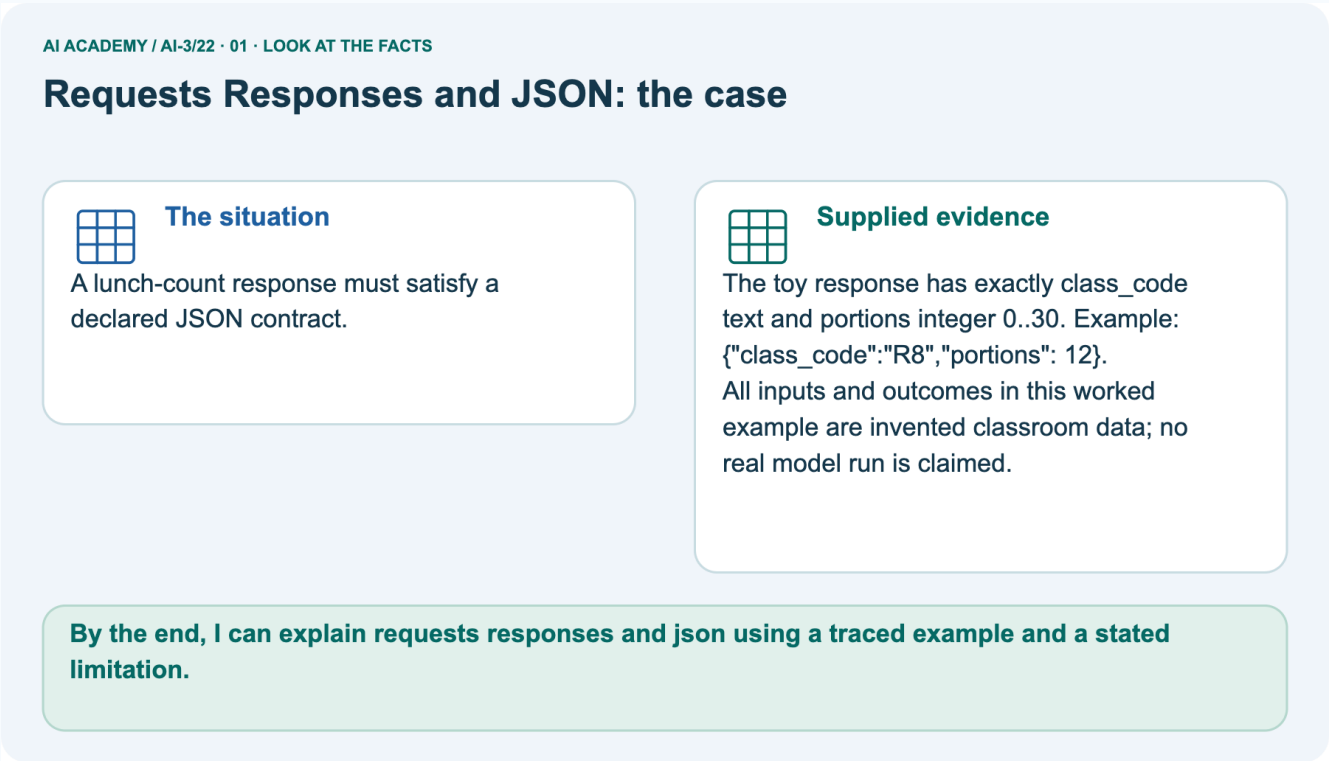
Keep your completed work

▼ Reference: the six-picture journey

Reference: the six-picture journey

> Revisit the picture that serves the current question; key diagrams already sit beside the main explanation.

01. Requests Responses and JSON: the case



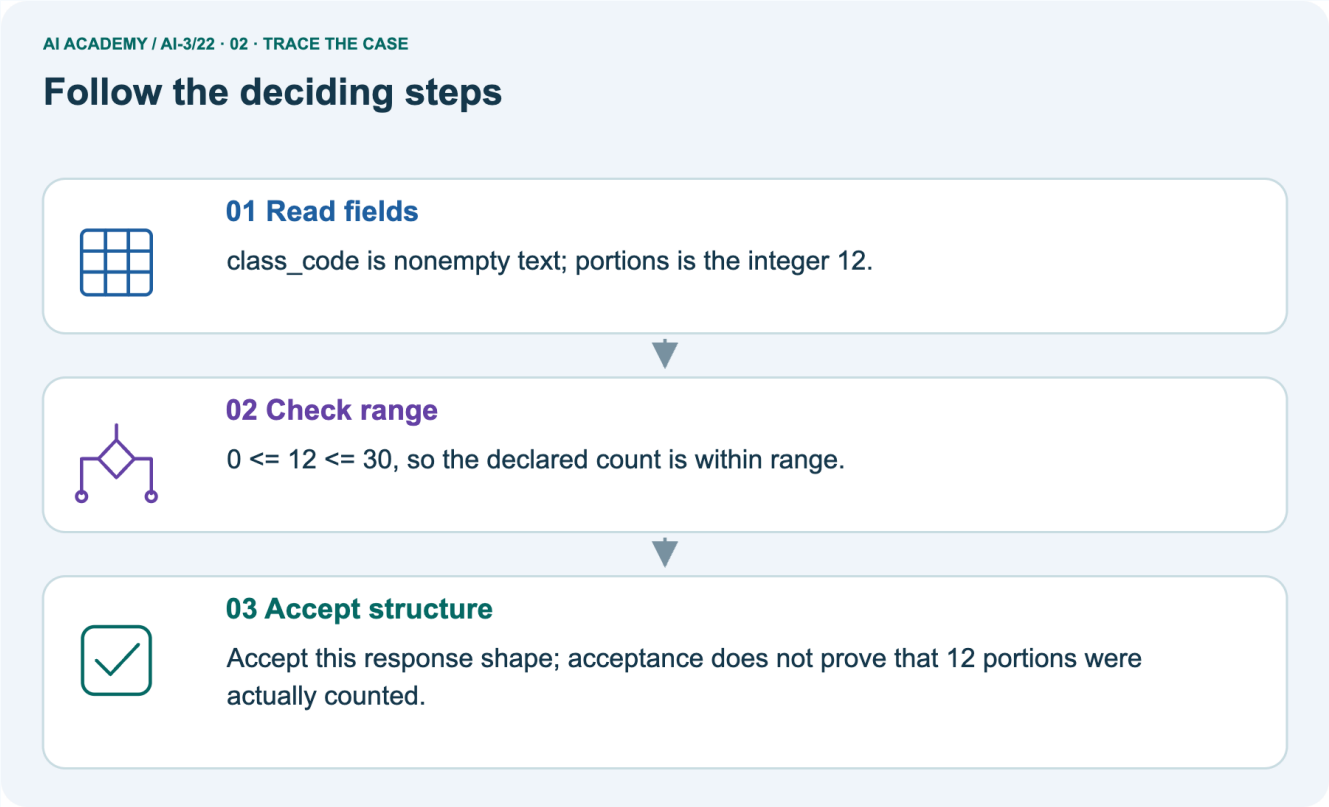
Follow the labelled picture with the matching explanation. Open the vector diagram

A lunch-count **response** must satisfy a declared **JSON contract**. Supplied facts: The toy **response** has exactly class_code text and portions integer 0..30. Example: {"class_code":"R8","portions": 12}.

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

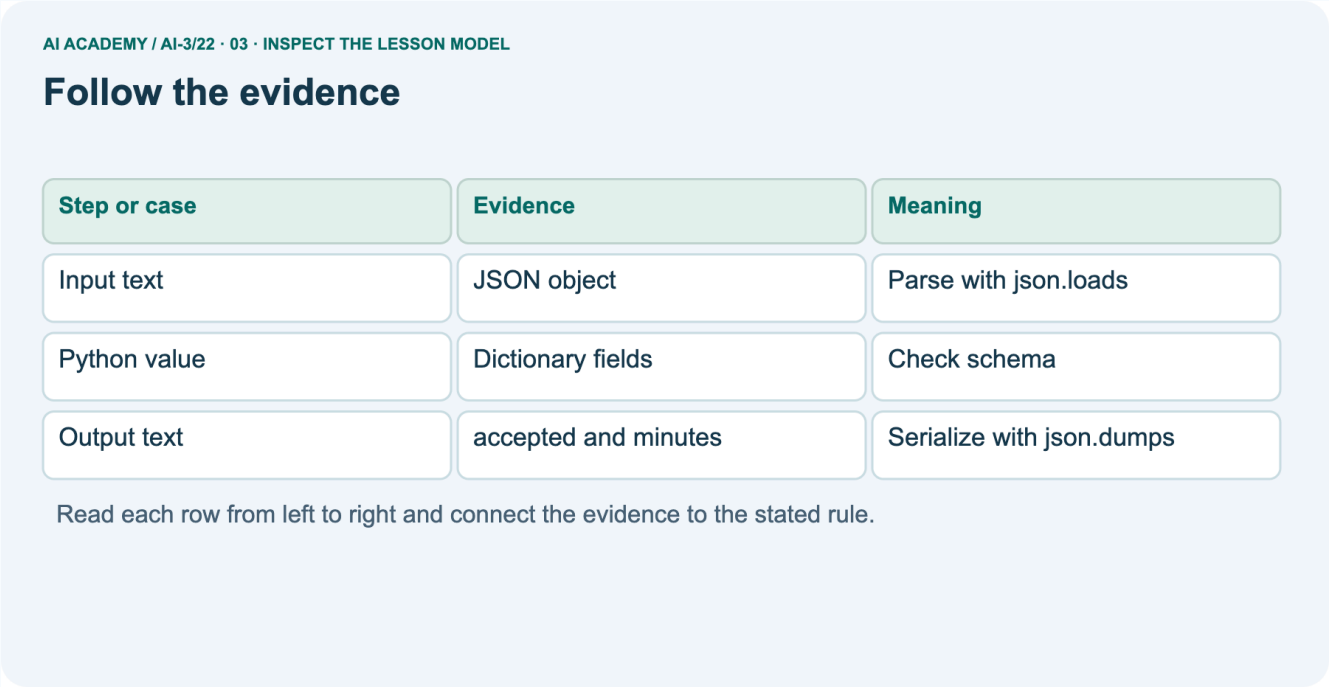


Follow the labelled picture with the matching explanation. Open the vector diagram

Read fields: class_code is nonempty text; portions is the integer 12. → Check range: 0 <= 12 <= 30, so the declared count is within range. → Accept structure: Accept this **response** shape; acceptance does not prove that 12 portions were actually counted.

Point to each numbered stage and explain which recorded **input** it uses.

03. Follow the evidence

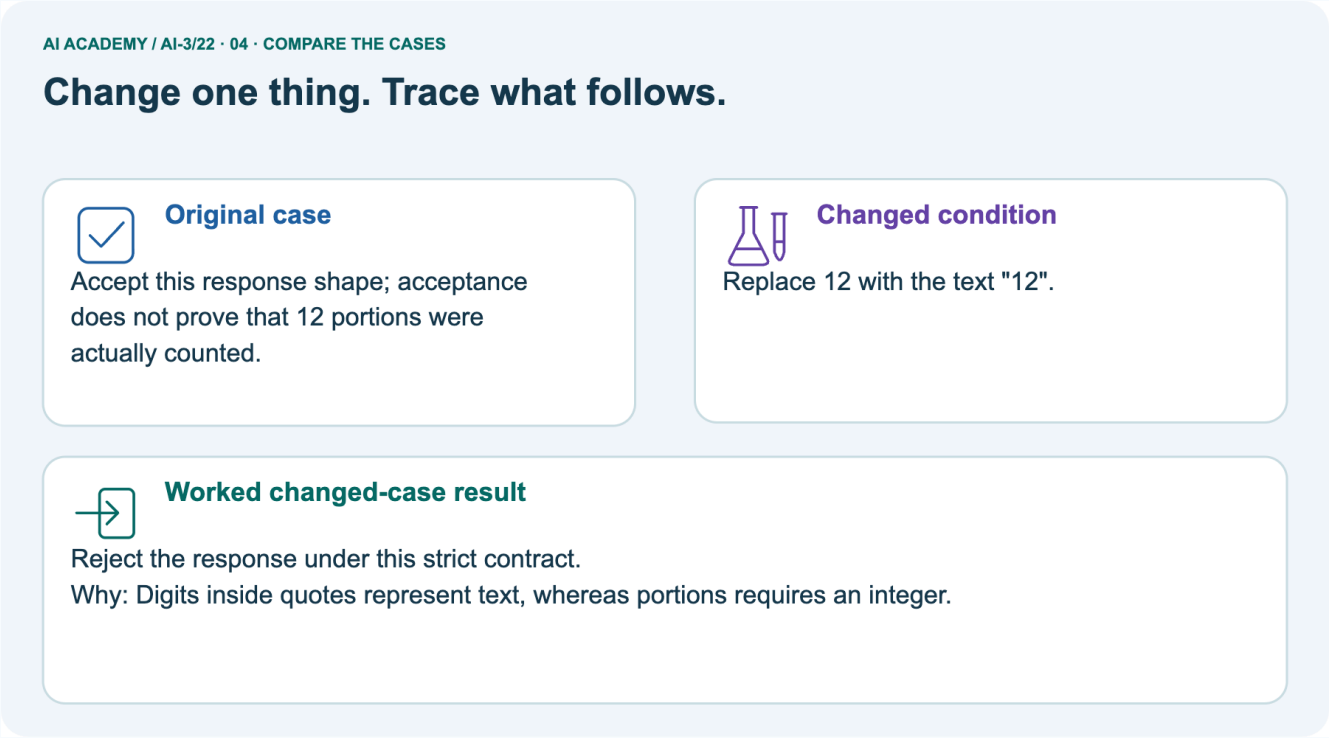


Follow the labelled picture with the matching explanation. Open the vector diagram

Follow the **evidence**. **Input** text | **JSON object** | Parse with **json**.loads. Python value | Dictionary fields | Check schema. **Output** text | accepted and minutes | Serialize with **json**.dumps

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



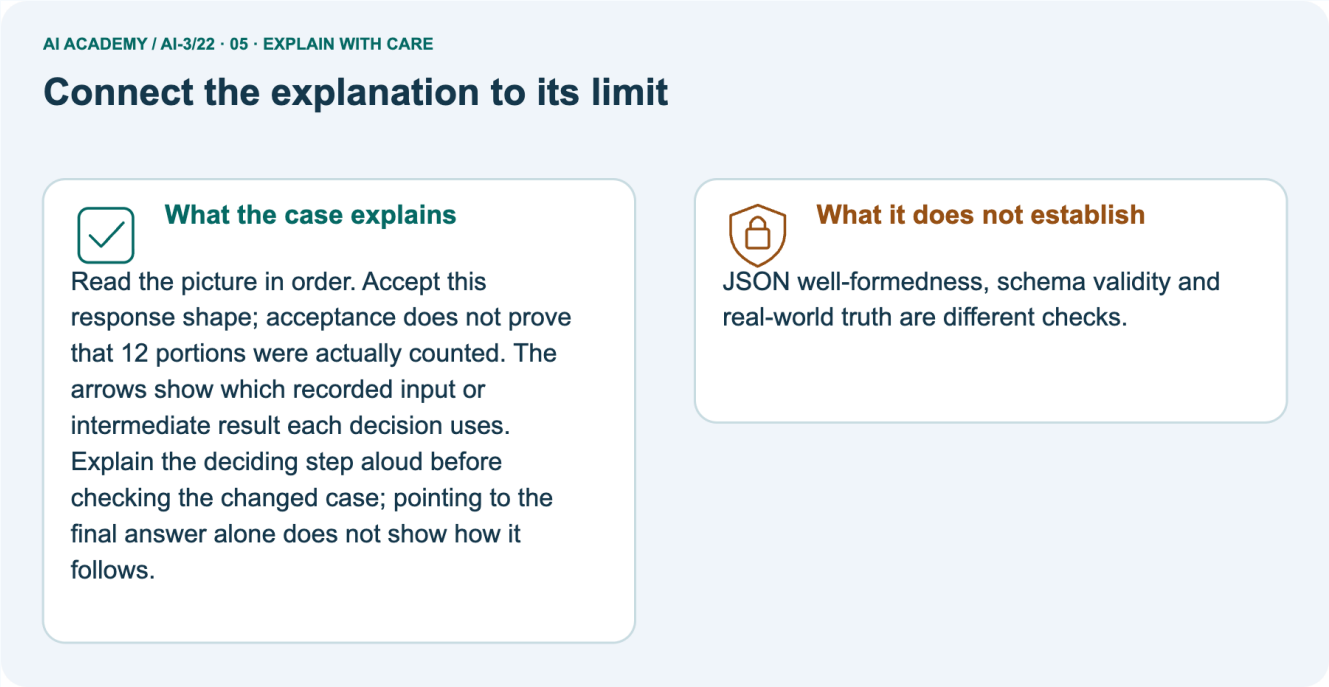
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Accept this **response** shape; acceptance does not prove that 12 portions were actually counted. Changed condition: Replace 12 with the text "12". Expected result: Reject the **response** under this strict **contract**.

Reason: Digits inside quotes represent text, whereas portions requires an integer.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



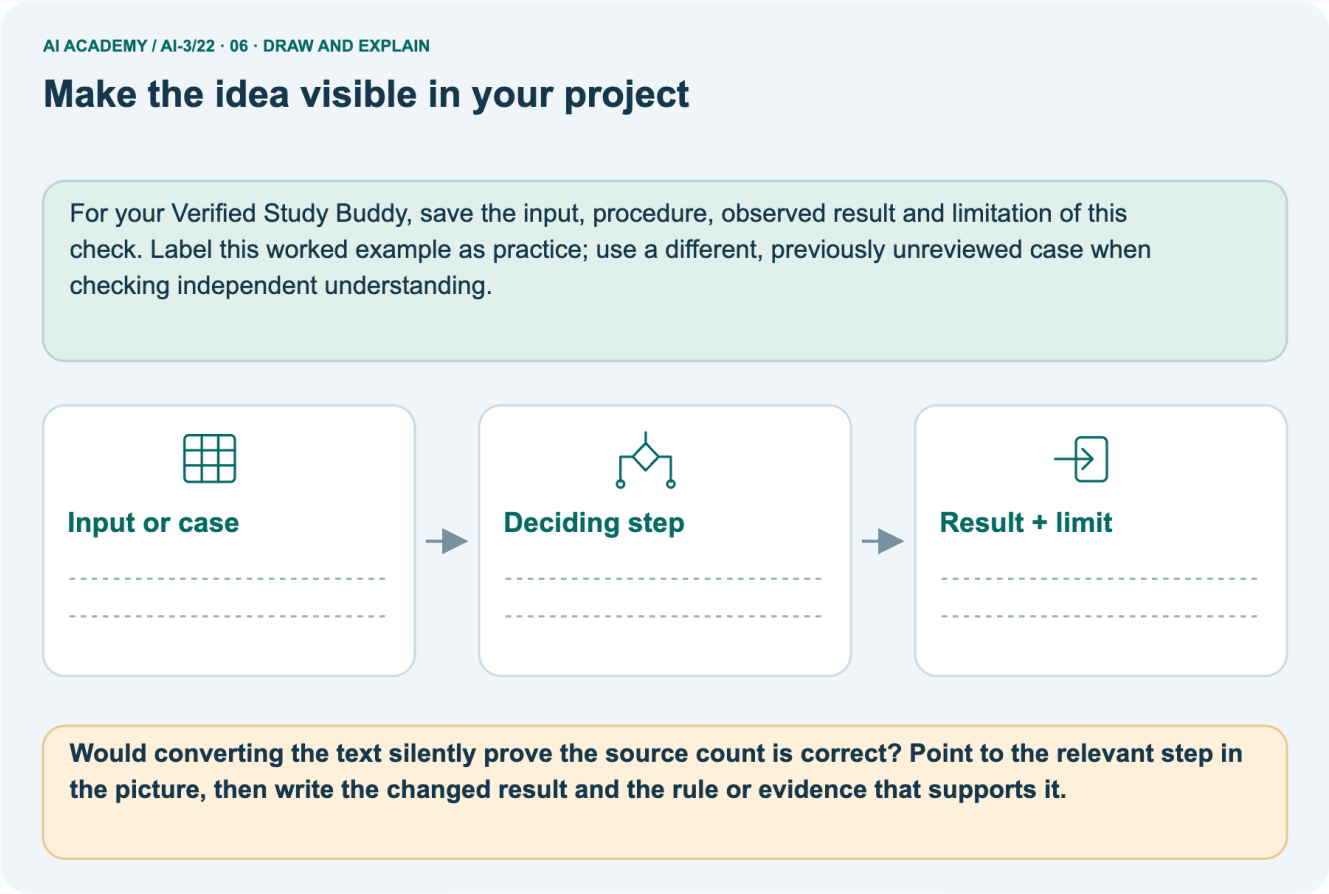
Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture in order. Accept this **response** shape; acceptance does not prove that 12 portions were actually counted. The arrows show which recorded **input** or intermediate result each decision uses.

Explain the deciding step aloud before checking the changed case; pointing to the final answer alone does not show how it follows. Limit: **JSON** well-formedness, schema validity and real-world truth are different checks.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For your Verified Study Buddy, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding. Draw your **input**, deciding step, result and limit.

Supported question: Would converting the text silently prove the source count is correct? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Requests Responses and JSON: the case

02. Follow the deciding steps

03. Follow the evidence

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.